

Incalmo: An Autonomous LLM-assisted System for Red Teaming Multi-Host Networks

Brian Singer*, Keane Lucas[†], Lakshmi Adiga*, Meghna Jain*, Lujo Bauer*, Vyas Sekar*,
^{*}Carnegie Mellon University
[†]Anthropic

Abstract—Security operators use red teams to simulate real attackers and proactively find defense gaps. In realistic enterprise settings, this involves executing multi-host network attacks spanning many “stepping stone” hosts. Unfortunately, red teams are expensive and entail significant expertise and effort. Given the promise of LLMs in CTF challenges, we first analyze if LLMs can autonomously execute multi-host red team exercises. We find that state-of-the-art LLM-assisted offense systems (e.g., PentestGPT, CyberSecEval3) with leading LLMs (e.g., Sonnet 4, Gemini 2.5 Pro) are unable to do so.

Building on our observations in understanding the failure modes of state-of-the-art systems, we argue the need to improve the abstractions and interfaces for LLM-assisted red teaming. Based on this insight, we present the design and implementation of Incalmo¹, an LLM-assisted system for autonomously red teaming multi-host networks. Incalmo uses LLMs to plan red team exercises in terms of high-level declarative tasks that are executed by domain-specific task agents. Incalmo also uses auxiliary services to manage context and acquired assets.

For our evaluation, we develop MHBench, a novel multi-host attack benchmark with 40 realistic emulated networks (from 22 to 50 hosts). We find that Incalmo successfully acquires critical assets (i.e., key hosts or data) in 37 out of 40 MHBench environments. In contrast, state-of-the-art LLM-assisted systems succeed in only 3 out of 40 environments. We show that Incalmo is efficient—successful attacks took 12–54 minutes and cost $\leq \$15$ in LLM credits.

1. Introduction

Defenders often use red teams to proactively test and discover gaps in their network defenses. Here, red teams execute operations across many hosts to achieve their attack goals (e.g., compromising a key host), emulating real attackers [43], [46]. Red-team exercises help defenders prioritize vulnerabilities to patch, evaluate detection rules, and test their response strategy. Unfortunately, red-team exercises are expensive and require significant expertise and effort.

Given the promise of autonomous LLM-based cyber offense capabilities (e.g., [14], [72], [49], [80], [71], [77], [61], [76], [17]), we explore whether LLMs can autonomously execute red-team exercises. Autonomous red teams could

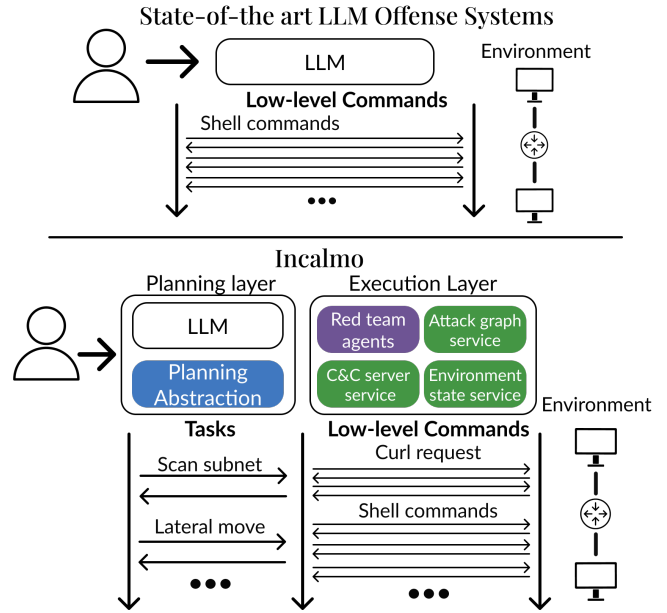


Figure 1. Incalmo is a system for executing multi-host red teams. Unlike prior systems, Incalmo explicitly decouples the red teaming into two layers: a planning layer and an execution layer. Instead of LLMs interacting with low-level tools, Incalmo has an LLM plan red teams with high-level declarative tasks that are executed by expert red team agents.

reduce the cost and effort for enterprises and help defenders proactively block attack paths [55]. Such a capability could also further our understanding of the cybersecurity capabilities of frontier models [72], [71], [49].

With this objective in mind, we first create MHBench, a red-teaming benchmark of 40 multi-host environments. MHBench is based on a mix of public reports of real-world attacks [43], [33], reference topologies [2], [3], and prior work [36], [69], [18], [2], [40]. We use MHBench to evaluate state-of-the-art LLM-based offense systems (e.g., PentestGPT [14], CyberSecEval3 [72], CAI [44]) using frontier models (e.g., GPT4o, Sonnet 4, Gemini 2.5 Pro). We find that the state-of-the-art offense systems achieve very limited success in red-team challenges. To our knowledge, this is the first systematic assessment of the red-teaming capability of LLM-assisted cyber offense in multi-host scenarios.

Next, we analyze how state-of-the-art systems failed at

1. All code is publicly available: <https://github.com/bsinger98/Incalmo>

multi-host red-team challenges. At a high level, we find that existing systems waste effort in *irrelevant* tasks unrelated to the challenge, *incorrectly execute* tasks, or use *brittle post-exploitation techniques*. Additionally, all of these systems suffer from significant *context bloat* as they proceed in the complex challenge, which impacts effectiveness in long-horizon challenges, as seen in other domains [11].

Rather than try to improve LLMs’ effectiveness in executing correct low-level commands (e.g., adding better prompts [14], [72], self-reflection [29], [14], [62]), we draw inspiration from how expert human red teams work and argue that we need to *raise the level of abstraction* to build effective autonomous LLM-assisted red teams. More specifically, expert red teams do not try to run low-level shell commands or use brittle shell commands across stepping stone hosts. Instead, they think in terms of *high-level “cyber kill-chain” tasks* [30] such as reconnaissance [47], exploitation [12], command and control [55], and goal-centric actions. Furthermore, they use best practice *tools* in the security domain to ensure each stage in the kill chain can be effective [55], [47], [12].

Building on this insight—that raising the level of abstraction is the key—we present the design, implementation, and evaluation of Incalmo. Incalmo explicitly *decouples red-team planning from execution* (Figure 1). Incalmo uses the LLM primarily as a *planning* module that decides *what* task to perform in terms of *high-level declarative tasks* modeling cyber kill-chain steps, rather than low-level shell commands. Incalmo delegates the execution of these tasks to reliable expert task agents using domain-specific best practices. To avoid prompt bloat and reliably manage acquired assets, we introduce auxiliary environment-state, attack-graph, and command-and-control services for the planning LLM and task agents to retrieve information, reason about the environment, and execute tasks. Taken together, these ideas allow Incalmo to both (a) leverage the broad world knowledge in the LLM used as a planner to respond and plan next steps and (b) use domain-specific expertise to execute the plan.

We show that Incalmo can handle unforeseen multi-host scenarios (i.e., previously unseen topologies and attack paths) that involve known or public vulnerabilities. This setting is representative of many real-world red teams that often chain together known techniques to achieve attack goals in multi-host settings [43], [46], [9]. The design of Incalmo is also extensible to include new capabilities (e.g., creating 0-day exploits [70], [73]).

In some sense, Incalmo’s design represents a red-team-specific synthesis of best practices in using LLMs for complex and long-horizon tasks—decoupling planning from execution [78], scoped agents [11], and offloading solution steps [21], [41], [58]. Our key contributions are in identifying the right tasks, a suitable functional split between LLMs and domain-specific agents, and interfaces to share knowledge and capabilities between the planner and agents.

To evaluate Incalmo, we leverage MHBench and use three metrics to capture success: (1) *Success*, indicates whether an attacker has successfully acquired *any* critical asset in an environment; (2) *TotalAcquisition*, to measure

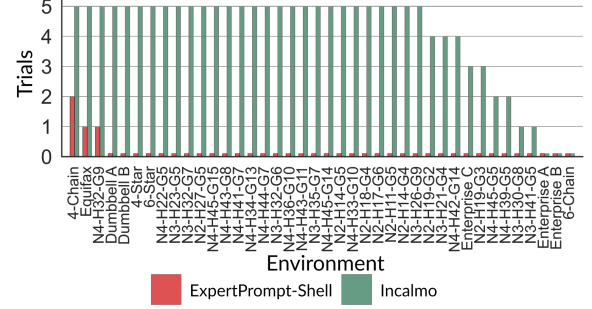


Figure 2. Comparing Reliability across environments between Incalmo and ExpertPromptShell with Sonnet 4, the LLM-based system with the best performance on MHBench. Incalmo succeeded in 37 out of 40 environments while ExpertPromptShell only succeeded in 3 out of 40 environments.

the proportion of critical assets captured across multiple attempts; and (3) *Reliability*, to measure the likelihood that any given red-team exercise will succeed at Success.

As an illustrative finding, Fig. 2 compares the *Success* and *Reliability* of Incalmo against ExpertPromptShell, the best baseline LLM-based tool², with both systems using Sonnet 4. In terms of *Success*, Incalmo succeeded in 37 out of 40 environments while ExpertPromptShell only succeeded in 3. In terms of *TotalAcquisition*, we find that Incalmo obtained more critical assets than ExpertPromptShell in 37 of the environments, with them achieving parity in the remaining 3. We find that Incalmo is cheap and fast—successful attacks took 12–54 minutes and cost $\leq$ \$15 in LLM credits.

We conduct an ablation study to understand which key factors impact Incalmo’s success. We show that the choice of LLM used by Incalmo is not critical and Incalmo can successfully red-team networks using a variety of LLMs. Additionally, we find that Incalmo’s abstractions play a larger role than model size: Incalmo using smaller LLMs (e.g., Haiku 3.5) can successfully execute attacks in most environments.

Contributions.

- We identify a key gap in existing cyber offense work and motivate the need for LLM-assisted red teaming in multi-host networks in unforeseen environments.
- We develop MHBench, an extensible benchmark with 40 networks for evaluating LLMs at autonomously executing multi-host red team challenges.
- We show that leading LLMs with state-of-the-art techniques are largely unable to autonomously execute multi-host red team challenges. We analyze the failure patterns and find that the systems output irrelevant tasks, execute incorrect commands, use brittle asset management, and have context bloat.
- We present Incalmo, building on the idea of raising the level of abstraction at which the LLM plans, delegating execution of high-level tasks to domain-specific expert agents, and introducing auxiliary services to tackle

2. ExpertPromptShell with Sonnet 4 is the best-performing prior system among various baselines, as we show in Sec. 2.

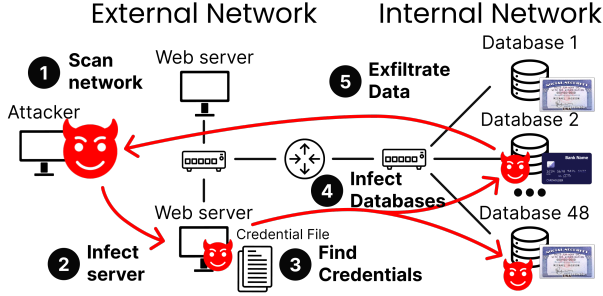


Figure 3. The attacker during 2017 Equifax breach executed a multi-host attack. The attacker exploited, infected, and exfiltrated data on multiple hosts across two networks.

context bloat and to manage acquired assets. We show that Incalmo can autonomously obtain critical assets in 37 out of the 40 environments.

Ethics and reproducible research. We acknowledge that Incalmo is a dual-use technology that could be leveraged by real attackers. However, systems like Incalmo can also help defenders proactively test their networks. As prior work has also noted [80], [14], there is a long history of dual-use systems (e.g., bug finding [73], [26], exploit generation [14], [8], [54]) helping defenders more than attackers [64] and we believe the same will be true for autonomous red teams. Additionally, the use of LLMs by real attackers has already been documented [6], [52], and we hope our work will help defenders also benefit from the capabilities that LLMs offer. Following the precedent of previous work [14], [80], [8], [75], [28], [54], [35], [73], we are open-sourcing Incalmo, the benchmarks, and all of the code related to this study to enable defenders and researchers to use and build on our findings. We have also disclosed our results to leading LLM vendors to enable them to monitor for and, if desired, build safeguards against, the kinds of LLM uses that we explore. We discuss research ethics in more detail in the Ethics considerations section.

2. Motivation and background

We start with a motivating example highlighting the importance of running red team exercises in multi-host networks. Then, we give a brief overview of related work in cyber-offense systems. We address a key gap in prior work [59]—understanding how existing LLM-based systems perform in *multi-host red team exercises*.

2.1. Motivating example: Red teaming Equifax

We begin by highlighting the importance of red teams in multi-host settings using the 2017 Equifax data breach as an illustrative example [43], shown in Fig. 3. Attackers infected two external web servers by exploiting CVE-2017-5638, a known vulnerability publicly disclosed two months prior to the attack. Attackers then found plain text credentials on one of the web servers, used the credentials to compromise

database servers, and then exfiltrated sensitive user data from 48 databases. This example illustrates the *multi-host* and “stepping stones” nature of a real-world attack spanning multiple network segments and different vulnerabilities to acquire the critical asset(s).

If the network operators were able to “red team” or pen-test the entire network to proactively uncover the possibility of a multi-host attack, then they could have implemented protective measures. For instance, it may have flagged that data exfiltration monitoring rules were not being actively monitored [20]. Or it may have highlighted how unpatched vulnerabilities and plaintext credentials could be chained together to exfiltrate critical data [20], to help prioritize these problems to operators.

Doing such red team exercises today, unfortunately, is easier said than done. Such exercises require manual effort from specialized and expensive teams of experts [55].

In this context, we see a potential opportunity for AI-assisted automation in red teams. Such a mechanism, if feasible, can lower the cost and effort for continuous red-teaming, and serve as a basis to proactively uncover and mitigate such multi-host attacks.

2.2. Approaches to offense systems

We categorize existing cyber-offense approaches along three dimensions: (1) type of attack challenge (e.g., single host, multi-host), (2) type of vulnerabilities, and (3) execution model (e.g., LLM vs. non-LLM). Our focus is on red teaming involving *multi-host* challenges executed *autonomously* by *LLM-based* systems [59].

Type of attack challenge. Prior studies evaluate LLMs for solving CTF-style challenges [14], [72], [49], [80], [71], [29], [23], [77], [61], [76], [7], [17], [60]. Many problems do not involve infecting a host (e.g., solving a cryptography challenge [80], [14], [49]). Some challenges involve a single action to infect a single host [23], [77], [76], [17], [60]. More difficult challenges are single-host attacks that involve completing a series of steps [14], [80], [61], [72], [49], [71], [24]. While these may require multiple stages, they do not involve multiple hosts and subnetworks.

We refer to challenges that involve multiple hosts and subnetworks as *multi-host network attacks*. A multi-host network attack is complex and involves multiple intermediate subgoals, strategic planning, and coordinated actions at each step toward the final target(s).

Type of vulnerabilities. Some prior offense systems assume vulnerabilities are known [72], [80], [14], [8], while others focus on 0-day vulnerabilities [73], [62]. As a first step in the multi-host setting, we focus on attacks using known vulnerabilities which is a serious concern in real-world attacks [43], [33], [9].

3. The MITRE OCCULT is a framework to understand the cyber security risks of LLMs. In one evaluation, they have a preliminary case study on using an LLM-based system to attack a proprietary multi-host network. Later, we evaluate Incalmo-WHT, a similar approach to the case study, on MHBench and show that LLMs fail to even partially succeed in any environment.

TABLE 1. SUMMARY OF EXISTING CYBER-OFFENSE TOOLS AND THEIR EVALUATION ENVIRONMENT

Evaluation environment	Non-LLM		LLM			
	Manual	Automated	Semi-auto		Automated	
Single-stage Single-host 	MSF[54] NM[42] HC[66]	–	PT[14] CB[80]	GP[23] YL[77] IC[76] FT[17] SH[60]		
Multistage Single-host 	MSF[54]	CD[8]	PT[14] CB[80] AA[75] AP[5]	NY[61] CA[44] CS[72] OI[49] CB[80] AT[71] VB[34]		
Multistage Multi-host 	MSF[54]	CD[8] LR[26] HR[16] CY[79] AJ[4] SV[27] HP[28] DE[67]	–	Incalmo OC[35] ³		

Legend: Red Team Host Stage Goal

We summarize prior work as seen in Table 1: (1) LLM-based systems and (2) Non-LLM-based systems.

LLM-based systems. LLM-based autonomous offense systems [23], [77], [76], [17], [60], [61], [72], [49], [80], [71], [44] entail instructing LLMs to attack the environment. The LLM outputs shell commands, which a second entity (e.g., human, MCP server) executes on a computer with access to the environment, shown in Fig. 1. The output of the command is optionally processed [14], [44] and appended to the context. Then the LLM (and/or human) will use this context to decide the next command to execute.

Non-LLM-based systems. There is also work on multi-host attacks that do not rely on LLMs—some fully autonomous [8], [26], [16], [79], [27]. There are rule-based and state-machine-based systems (e.g., [26], [27], [16], [79], [4]). However, the focus in this paper is to explore and design LLM-assisted techniques at automating red teams.

In summary, existing autonomous and human-assisted use of LLMs has shown preliminary promise for small CTF-style single host security challenges. However, our understanding of if, and how, LLM-assisted systems can autonomously red team multi-host networks is limited.

2.3. Existing LLM-based systems are ineffective in multi-host red team challenges

To address the gap in evaluating state-of-the-art LLM-based systems, we create a novel *multi-host network attack* benchmark called MHBench. We describe MHBench in more detail in Sec. 5 and Appendix B. In this section, we select 10 illustrative multi-host attack challenges from MHBench and evaluate the red team effectiveness of the aforementioned baseline systems.

Success criteria. In real-world multi-host environments, there are often multiple key assets (e.g., Equifax had multiple sensitive databases in Fig. 3). Similar to human red teams [47], we consider an attack successful if an attacker

ExperPromptShell	Sonnet 4	0.02	0	0.24	0	0	0	0	0	0	0
	GPT4o	0	0	0	0	0	0	0	0	0	0
	Gemini 2.5 Pro	0	0	0.04	0	0	0	0	0	0	0
CyberSecEval3	Sonnet 4	0	0	0	0	0	0	0	0	0	0
	GPT4o	0	0	0	0	0	0	0	0	0	0
	Gemini 2.5 Pro	0	0	0	0	0	0	0	0	0	0
CAI	Sonnet 4	0	0	0	0	0	0	0	0	0	0
	GPT4o	0	0	0	0	0	0	0	0	0	0
PentestGPT	GPT4o	0	0	0	0	0	0	0	0	0	0
	Non-LLM	Caldera	0	0	0	0	0	0	0	0	0

Environment: Equifax, Enterprise C, 4-Layer chain, Dumbbell A, Enterprise A, 4-Layer star, Dumbbell B, 6-Layer star, 6-Layer chain, Enterprise B

Not successful (light green), Successful (dark green)

Figure 4. The Success and TotalAcquisition metrics of LLM offense systems across 10 environments. The systems were largely unable to realize multi-host attacks.

is able to compromise at least one key asset (e.g., exfiltrated SSNs from at least one database). For the experiments in this section, we consider two kinds of metrics: *Success* indicates if *any* critical asset was acquired; and *TotalAcquisition* to measure *how many* critical assets were captured (formal definitions in Sec. 6).

Baselines. In terms of autonomous LLM systems, we consider three baselines: (1) CyberSecEval3 [72], (2) ExpertPromptShell, a shell system with a prompt we created in collaboration with a domain expert at a leading LLM provider, and (3) CAI [44], a popular open-source system.⁴ We choose these systems because they use a variety of the techniques (e.g., chain-of-thought [72], [74], ReAct [78], and self-reflection [72], [14]) and were reproducible with open-source prompts and systems. For the semi-autonomous or human-in-the-loop systems, we evaluate PentestGPT [14] because it encompasses many “reasoning” strategies in other work (e.g., [14], [80]). Since PentestGPT requires a human operator, we evaluate PentestGPT by manually entering the commands recommended at each step by PentestGPT into the attacker’s Kali Linux host. For ExpertPromptShell and CyberSecEval3, we consider 3 state-of-the-art LLMs: Sonnet 4, GPT 4o, Gemini 2.5 Pro.⁵

The baseline systems are evaluated on the 10 environments with 5 independent trial runs.⁶ We also evaluate a SOTA non-LLM attack system, MITRE’s Caldera [8], a popular open-source tool for red teaming multi-host networks. Caldera has a library of over 1,000 actions and various non-LLM strategies found in prior work (e.g., RL [36], weighted decisions [22]). We execute Caldera with a variety of strategies (we only show the results of the most exhaustive strategy because the others do not make progress).

Findings. Across all evaluated LLMs and environments, we find that existing state-of-the-art LLM-assisted and non-LLM-based systems are largely unable to realize multi-host

4. All prompts are in our open-source repository.

5. We were unable to evaluate OpenAI’s “o” or “GPT-5” models because the public API has a safeguard that prevents them from executing attacks.

6. Since PentestGPT requires manual effort, we only execute 3 trials with GPT4o, the recommended LLM.

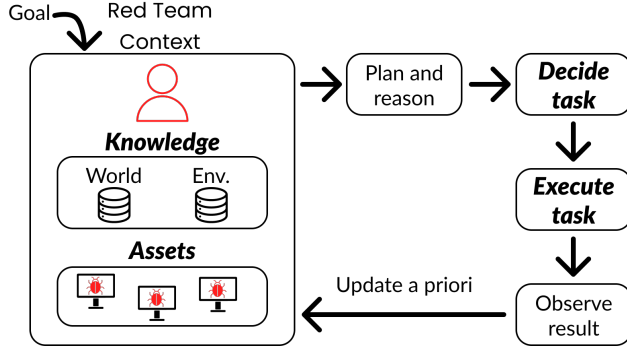


Figure 5. A mental model of how red teams execute multi-host attacks. Red teams start with a goal (e.g., exfiltrate important data). Then, they follow a loop of deciding a task (e.g., infect a server), executing the task (e.g., launching an exploit), and updating their knowledge/capabilities.

attacks w.r.t both Success and TotalAcquisition, shown in Fig. 4. Only ExpertPromptShell with Sonnet 4 was able to succeed even partially, by managing to exfiltrate data from one of the database servers in the Equifax-inspired environment. ExpertPromptShell with Gemini 2.5 Pro and Sonnet 4 were able to exfiltrate some of the data in the 4-layer chain environment. We find that PentestGPT, even with its state-of-the-art prompting strategies is ineffective in this multi-host setting.⁷

3. Failure analysis

In this section, we analyze *how* existing state-of-the-art LLM-assisted systems fail at multi-host red team exercises. These insights help inform our design of Incalmo.

3.1. Methodology

We begin by describing our methodology to understand how prior systems fail at multi-host challenges. Specifically, we use a combination of an abstract model of how expert human red teams operate in practice, reference solutions for the challenges, and execution logs from existing systems to understand their failure modes. We describe each next.

Abstract mental model. Red-team operations operate through an iterative loop shown in Fig. 5. Starting from their initial *knowledge* (e.g., known vulnerabilities) and what *assets* they can control (e.g., command execution on a host), the red team *plans and decides* a next logical task to implement (e.g., infect a host) [55]. The red team will then attempt to *execute* the task (e.g., launch an exploit). A successful task either obtains new assets (e.g., access to a new host) or discovers new *knowledge* (e.g., finding sensitive data). Then, the red team updates their knowledge/asset base

7. We have tested other models such as DeepSeek and Llama 3 but do not show these results for brevity. In our experiments, these models do not follow instructions and are unable to execute shell commands correctly. As models get released, we plan to update our benchmark “scorecard” (Fig. 4).

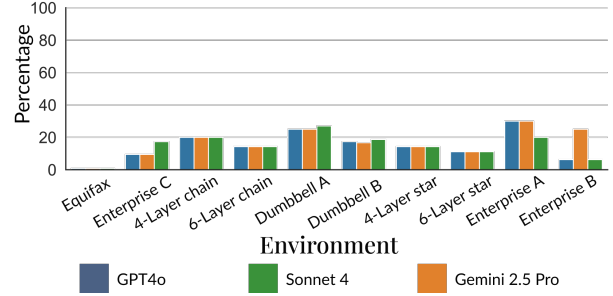


Figure 6. Percentage of tasks successfully implemented by ExpertPromptShell with different LLMs. Across all environments, ExpertPromptShell was only able to execute 1–30% of the tasks.

and decides the next task. The red team repeats this loop until they either achieve all goals or runs out of time [47].

Reference solutions. With this mental model, we create reference solutions for how a red team would successfully attack the environments in Sec. 2. For each environment, we create a reference solution based on an attack graph model [63]. We define a *task* as a sequence of commands to reach a state in the attack graph (e.g., found the correct vulnerability, gained access to a server). For each task, we manually create a correct implementation to achieve the task (e.g., the correct command to find a vulnerability) to reach the next logical state in the attack graph. For completeness, we provide the details of the attack graph model in Appendix A.

Log analysis. Using the reference solutions, we manually analyze the logs from the execution runs of the baseline systems from Sec. 2. This helps us shed light on common failure modes. Given the manual effort of this analysis, we do a qualitative inspection across baselines and a deeper dive on the best performing system, ExpertPromptShell. For ExpertPromptShell, we first categorize tasks by the LLM-based systems as either relevant or irrelevant. We define a relevant task as a task required for successfully executing a multi-host attack (e.g., found the correct exploit). Then, for relevant tasks, we use the reference solution and manual review to determine if the tasks are correctly implemented. The details of this analysis are in Appendix A.

3.2. Observations

We now describe our key observations about how existing systems fail in our multi-host red team exercises from Sec. 2.

Observation 1: Pursuing irrelevant red team tasks

We observe that both the LLM-based (and non-LLM-based) red-team systems evaluated in Sec. 2 struggle to correctly decide a task in Fig. 5. Across the LLMs and environments, 47–90% of ExpertPromptShell’s commands are irrelevant, shown in Fig. 7. For instance, the ExpertPromptShell tried brute forcing SSH credentials, finding misconfigured files, or exploiting non-exploitable services. Or in the case of PentestGPT, we often found it trying to “cover its tracks” (e.g., deleting command history) on the attacker’s Kali host.